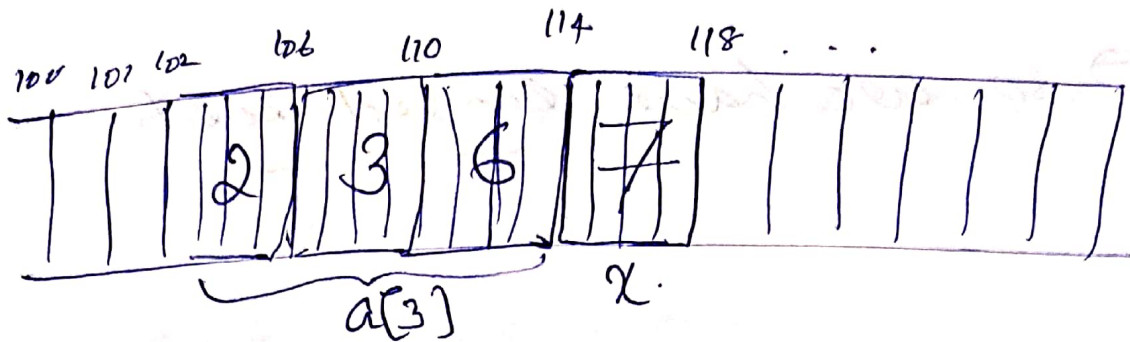


Linked List

Mem. located in Comp. Sys.



int $x = 7$; 4 bytes

int $a[3] \leftarrow$ 12 bytes

If you want to change $a[3]$ to $a[4]$

not possible
consecutive block not available.

So a fresh block of 16 bytes 4×4 has to be allocated

future (10) $\rightarrow$

$a[50]$

max-limit

mem: 50×4

200 bytes

use only 3 & 4 bytes at a time.

space is wasted : right ?

wastage of mem. $\rightarrow$ arrays

$\hookrightarrow$ so we have linked list

Array - collection of elements of same data type.

int a;

int a[4];

$\downarrow^4$

7	9	3	2
1000	1004	1008	1012

different.

depends on compiler.

turbo C $\rightarrow$ 2 bytes

most compiler $\rightarrow$ 4 bytes

continuous memory allocations.

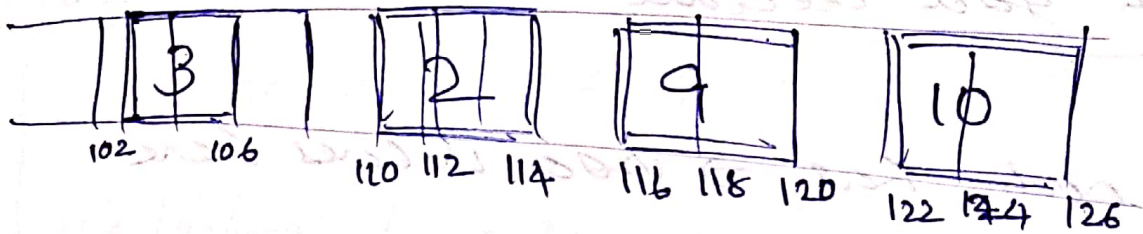
u. cannot change array size as runtime

7	9	4	3	2
1000	1004	1008	1012	1020

LL

consist of a node.

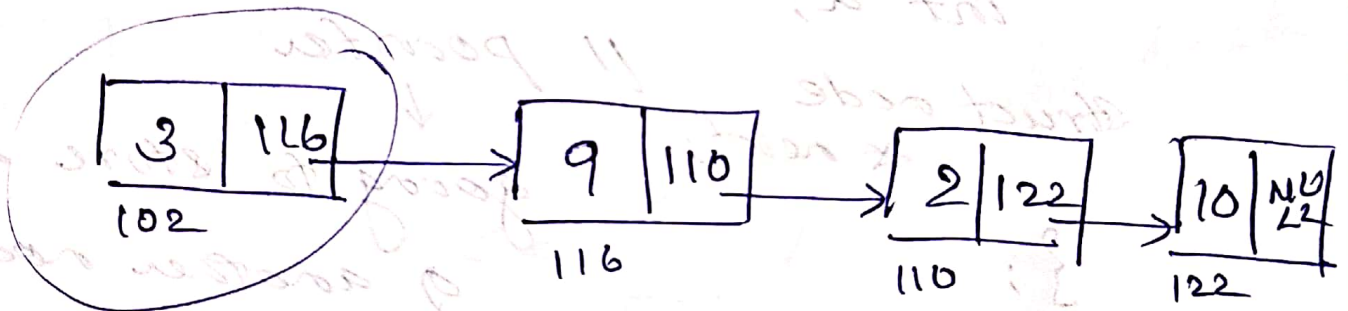




LL $\rightarrow$ stored in different locations

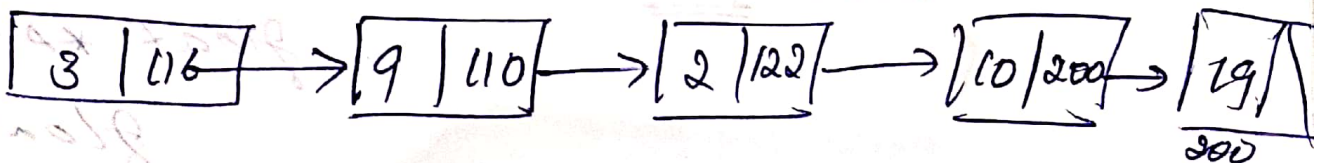
not consecutive.

but uses a pointer to point to next location. or address of next one.



complete is known as node.

if we want to insert 19.



How you declare LL.

A node has 2 values here

2 different types

int

pointer

So user-defined data type $\rightarrow$ Structure

struct node

{

int a;

struct node

* next;

};

// pointer

↓
going to store address
of another node.

struct node

data type

↓
like int x.

int *p

↓
stores address
of integer
variable

float *p

float variable

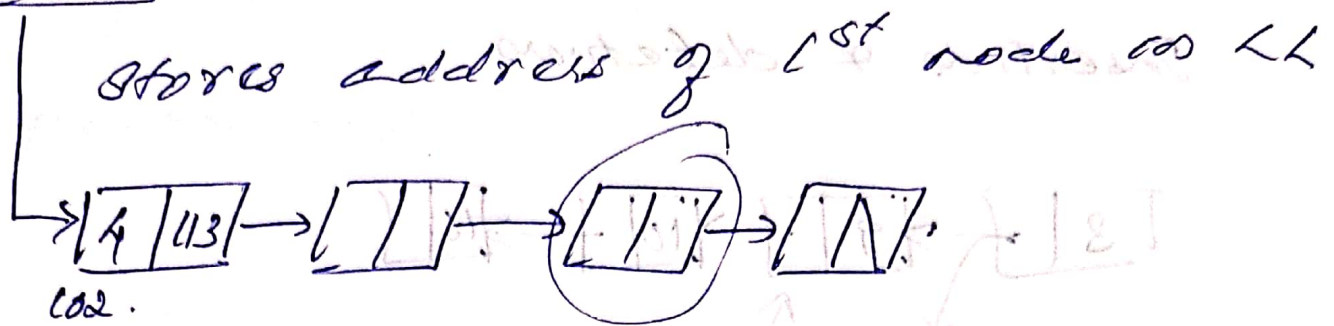
So here,

struct node * next

data type

pointing to

[102] head pointer



Cannot access directly

have to traverse.

in array $a[] \rightarrow$ will give
here not possible.

Data link.



draw back

So in LH $\rightarrow$ sequential access

traverse the list until you

reach there.

$O(n)$

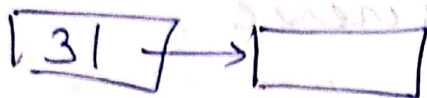
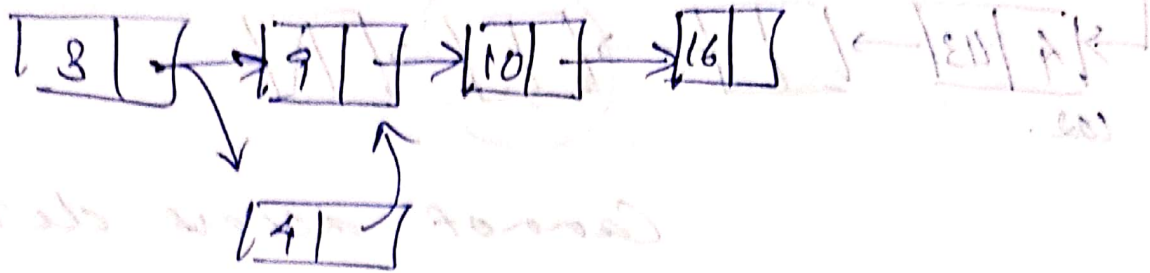
accessing of any element
(worst case)

best case $\rightarrow O(1)$ 1st element

In array $\rightarrow$ all $\rightarrow O(1)$

~~So, LL is~~

Insertions & deletions.



Similarly deletion is easier

Briefly

LL is a linear DS. Collection of data elements which are stored in non-continuous memory locations.

extra space required to store the pointers with each value

Insertions & deletions is easier than array.

Accessing of any element $O(n)$
array $O(1)$

Binary Search not possible in LL.

Dynamic size.

Types of LL

- ① Singly LL
- ② Doubly LL
- ③ Circular LL
- ④ Doubly Circular LL

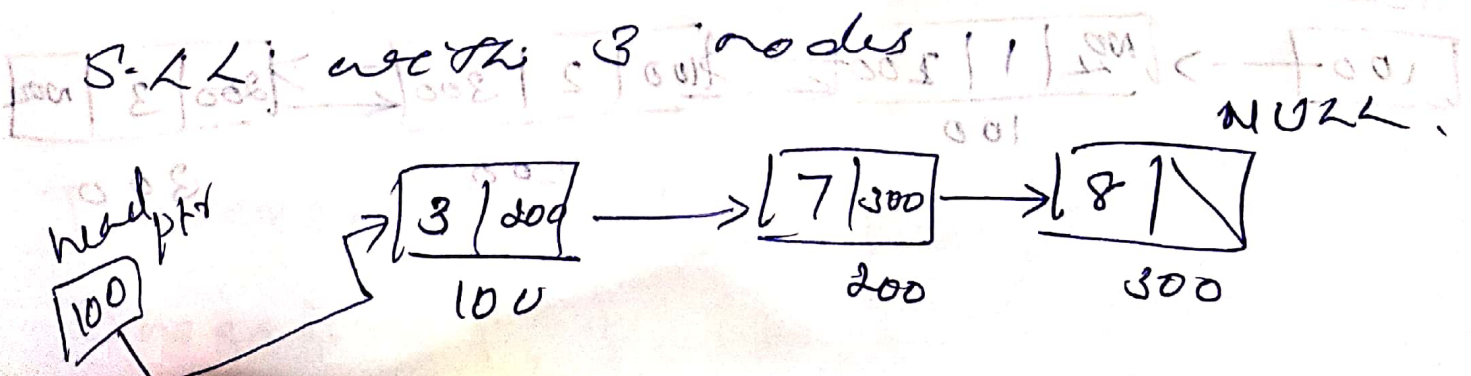
Singly LL — single length only.

— each node contains 2 parts
data part
address part.

& pointer for next node.

Node.

Data/addr



- only forward traversal possible.

struct node

{ int data;

struct node *next;

};

Doubly LL

2 pointers

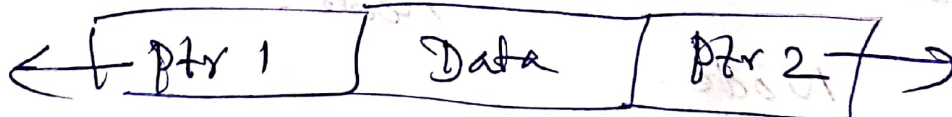
each node has 3 parts

1. data part.

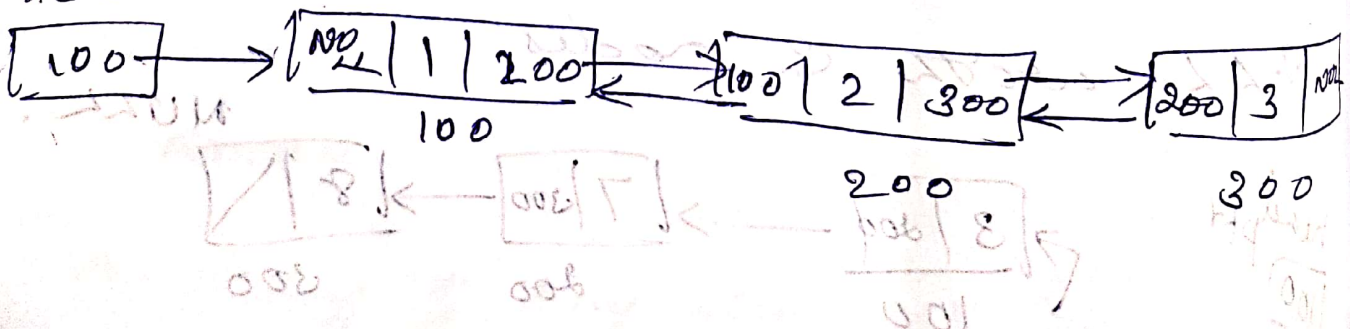
ptr to prev. node

ptr. to next node

Node.



Head



struct node

```
{ int data;
```

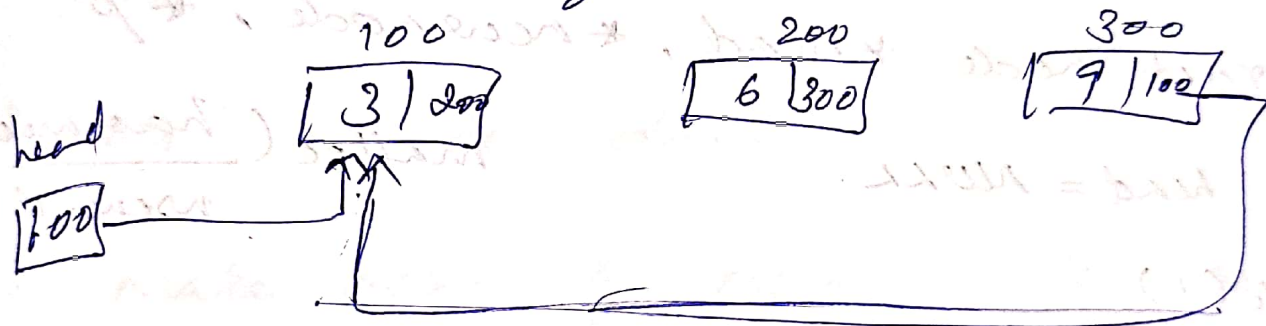
```
struct node * next;
```

```
struct node * prev;
```

```
};
```

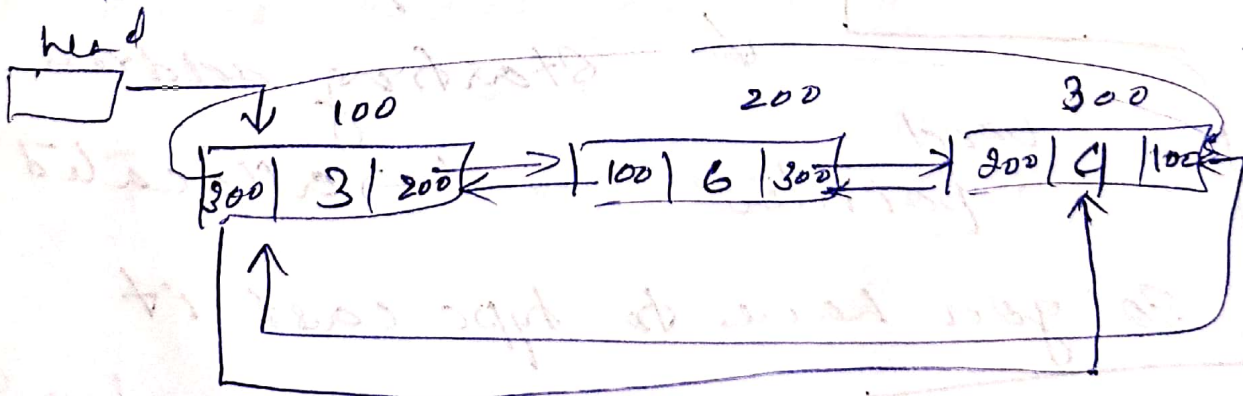
Circular linked list

variations of S-LH.



Doubly C-LH

last element is linked to first element in list.



Traversal
Insertion

Deletion.

Declaring a linked list.

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

Creating a node

```
struct node *head, *newnode, *p;
```

```
head = NULL;
```

malloc (how much
 mem. I want)

```
while (1)
```

```
{  
    newnode = (struct node*) malloc (size of (struct node))
```

returns pointer to the

starting address

↳ allocated

void
pointer

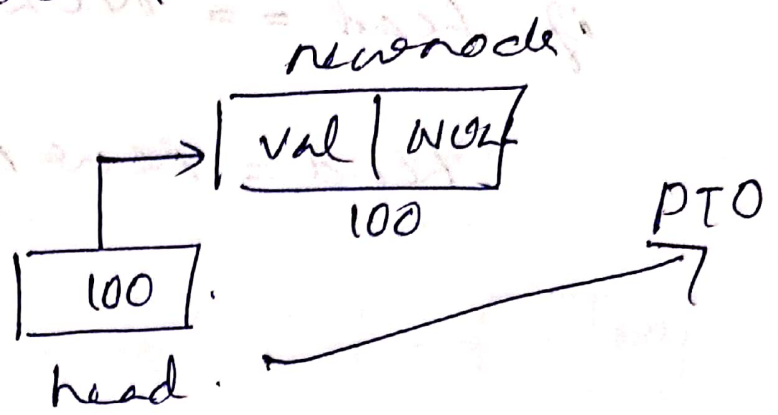
So you have to type cast it

newnode → data = value

newnode → next = NULL

val	NULL
-----	------

head = new node.



Inserting a node

Beginning / or at front.

1. allocate / create node.

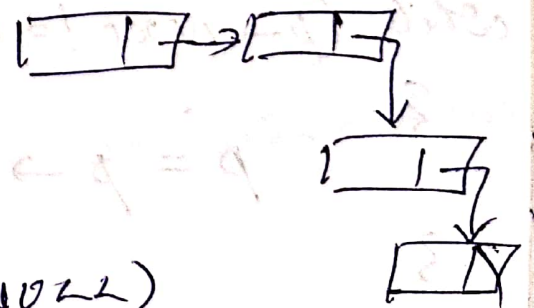
2. put in the data.

3. make next of new node as head

new node $\rightarrow$ next = head.

4. make head point to the new node.

Traversing the list



$p = \text{head}.$

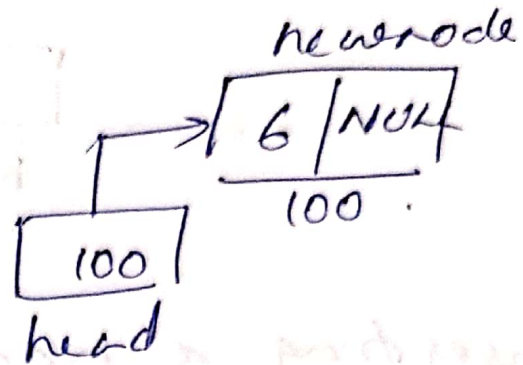
while ($p \rightarrow \text{next} \neq \text{NULL}$)

{ $\text{printf}(\text{"\%d"}, p \rightarrow \text{data})$

$p = p \rightarrow \text{next}.$

if head == NULL.

head = new node.



Again you will create a newnode

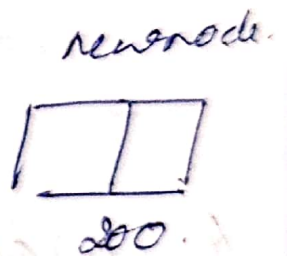
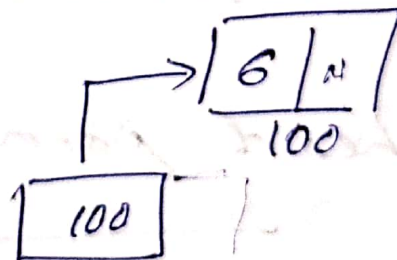
if (head == NULL)

head = new node.

else.

{

p = head.



while (p->next != NULL)

{

p = p->next

}

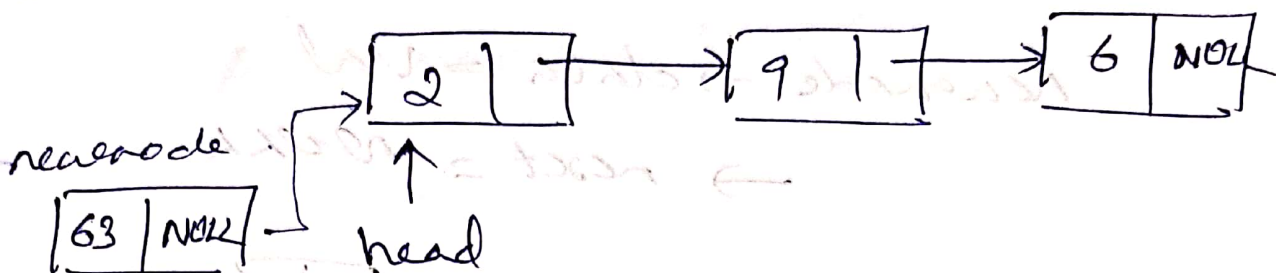
p->next = newnode.

newnode->data =
" -> next = N

} exit condition -> return while (1)

Insertions

beginning or at the front



$\text{newnode} \rightarrow \text{next} = \text{head}$

$\text{head} = \text{newnode}$

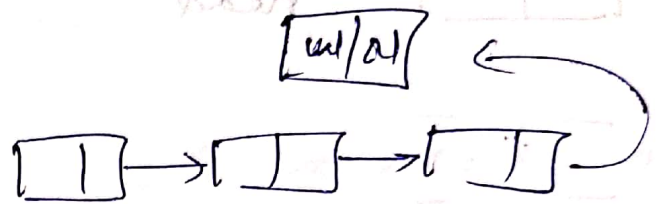
insert at end

1. allocate
2. put in the data.
3. $\text{newnode} \rightarrow \text{next} = \text{NULL}$
since it is the last node
4. If list is empty, "
make newnode as head
5. Else traverse till the last node

6. Change the next of last node to newnode.

newnode $\rightarrow$ data = val

$\rightarrow$ next = NULL



while ($p \rightarrow$ next $\neq$ NULL)

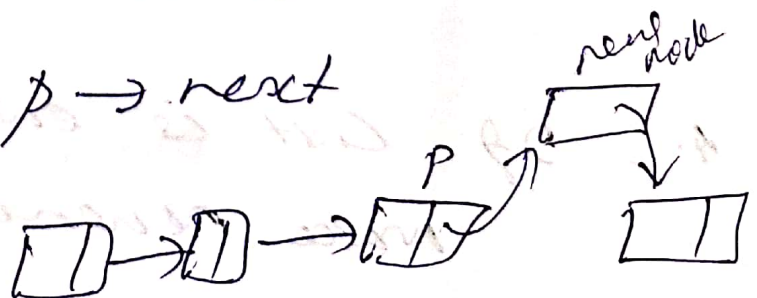
$p = p \rightarrow$ next

$p \rightarrow$ next = newnode.

Insert after a given node - val

while ($p \rightarrow$ data $\neq$ val)

$p = p \rightarrow$ next



~~new~~ ^P

new node $\rightarrow$ next = $p \rightarrow$ next

$p \rightarrow$ next = newnode.

Deletion

1. delete_beg()

if head == NULL
List empty

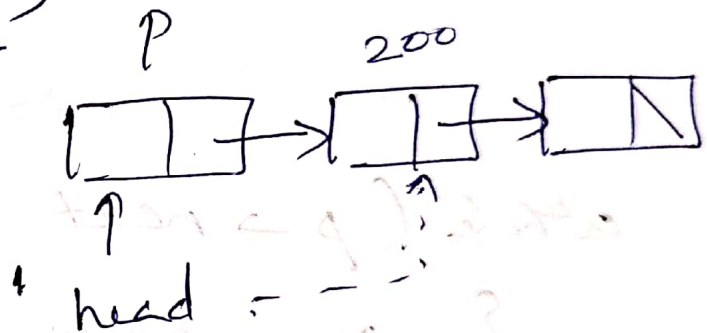
else

$p = \text{head}$.

$\text{head} = p \rightarrow \text{next}$.

$\text{free}(p)$

}



you have to
free that
memory also

2. delete_end()

——— 2 pointers
needed

$p = \text{head}$

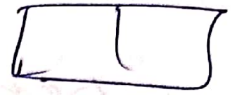
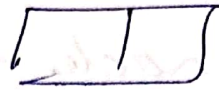
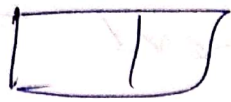
while ($p \rightarrow \text{next} \neq \text{NULL}$)

$p = p \rightarrow \text{next}$.

explain.

prev-node.

Prev node P.



delete from end $\rightarrow$ 2 pointers

$p = \text{prev-node} = \text{head}$



while ($p \rightarrow \text{next} \neq \text{NULL}$)

{ prev-node = p

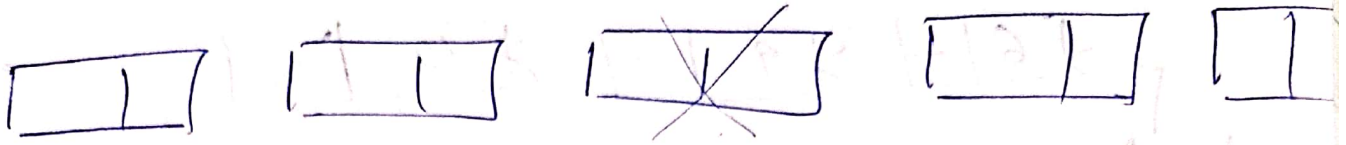
p = p $\rightarrow$ next

}

prev-node $\rightarrow$ next = NULL

free(p).

delete a particular node



prev-node

p

~~next-node~~

$p = \text{prev-node} = \text{next-node} = \text{head}$

while ($p \rightarrow \text{data} \neq \text{val}$)

{

prev-node = p

$p = p \rightarrow \text{next}$

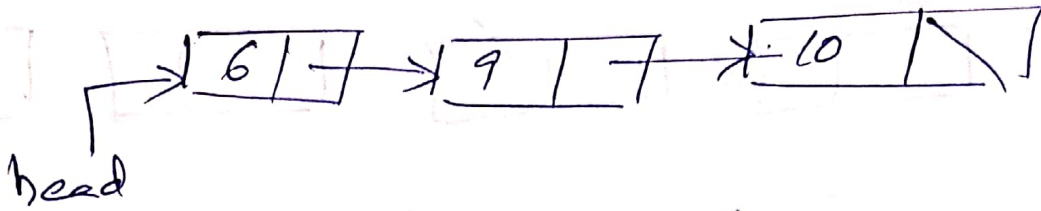
~~next-node = p → next~~

}

$\text{prev-node} \rightarrow \text{next} = p \rightarrow \text{next};$

free(p)

Stack using LL



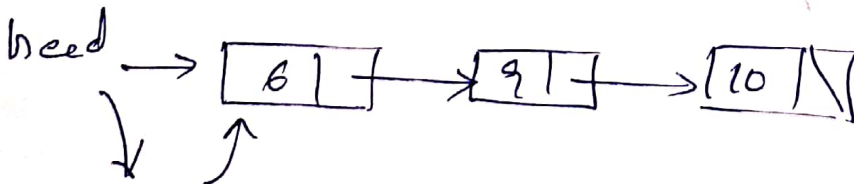
Stack - LIFO

2 options : ① Insert & delete at end } $O(n)$

push & pop } $O(1)$ ② Insert & delete at front.

Features of Stack

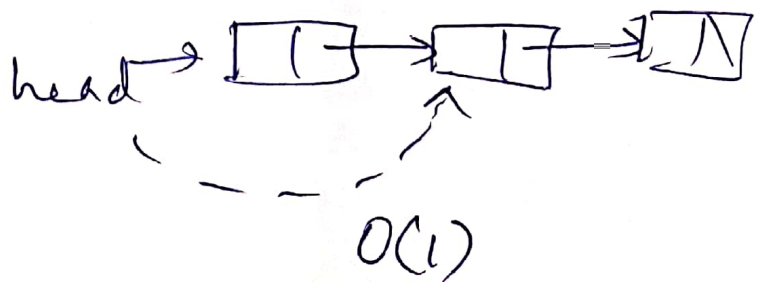
$O(1)$ → operations
LIFO.



Delete

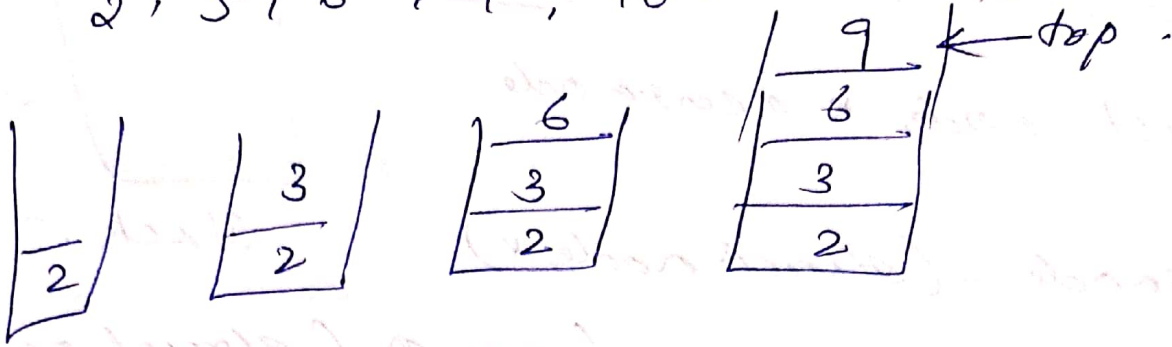
new node

$O(1)$

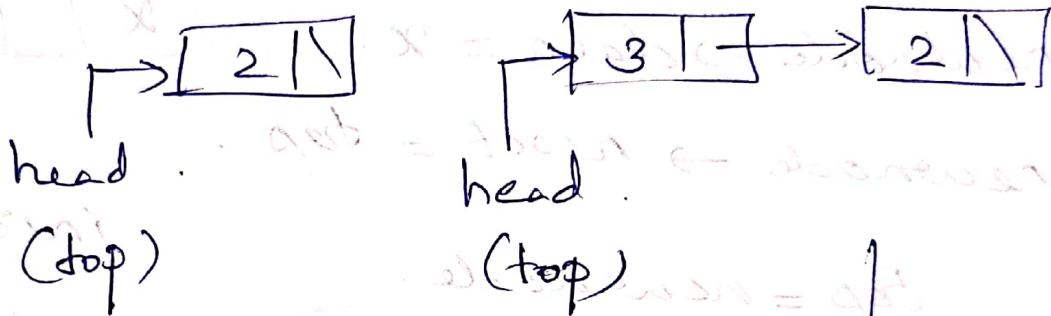


$O(1)$

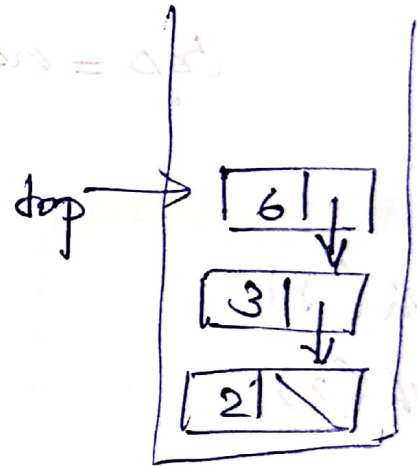
2, 3, 6, 9, 10.



LL



insert & delete at
front of
LL



struct node

{
int data

struct node *link

};

struct node *top = NULL.

push(x)

top
newnode



Stack.

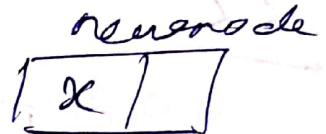
→ struct node * newnode.

→ newnode = (struct node *)

malloc (size of (struct node))

→

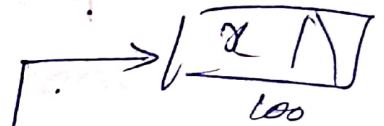
newnode → data = x.



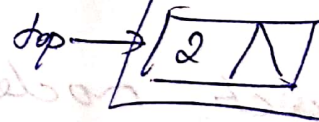
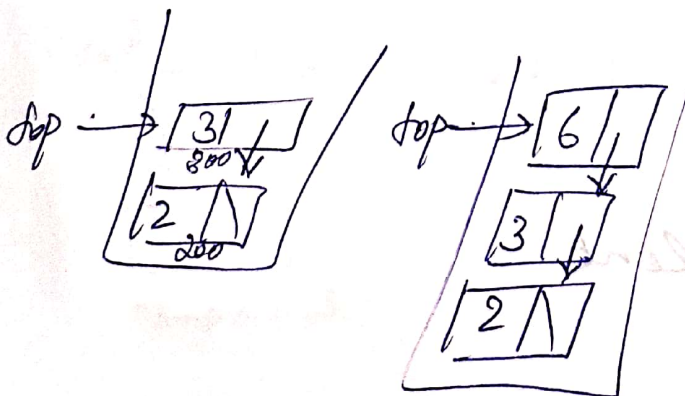
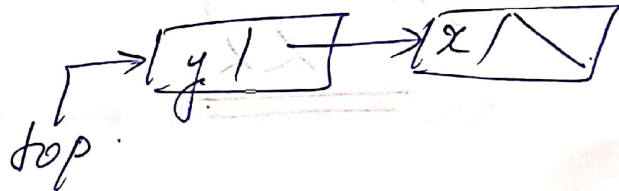
newnode → next = top.

top = newnode.

initially



push(2)
push(3)



push

no overflow condition

dynamic

allocation

display()

6 3 2

*temp

temp = top

while (temp->next != NULL)

printf("%d", temp->data)

temp = temp->next

if (temp == NULL)

List is empty

else

{

==

}

peek()

gives top element of stack.

if top == NULL

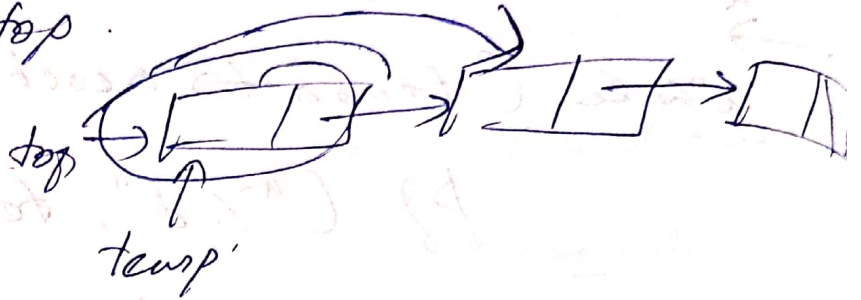
Stack empty

else printf("%d", top->data);

pop()

struct node *temp.

temp = top.



if top == NULL

underflow.

else.

popped element

printf("%d", temp->data)

top = temp->next

free(temp)

Queue using LL

FIFO

head

$O(1)$



front

deletion

$O(1)$

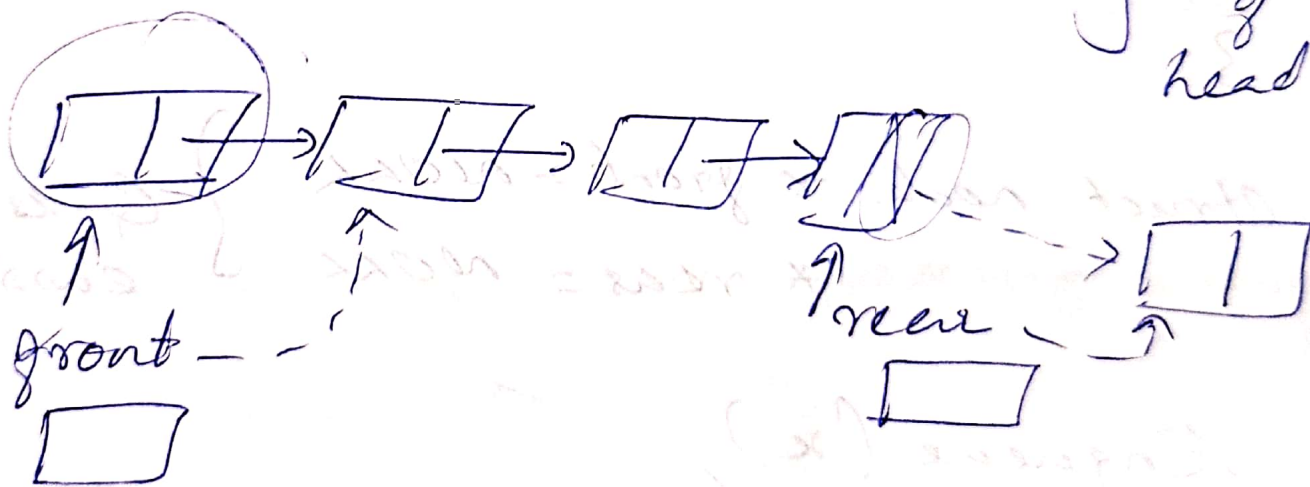
rear

inserting at here

$O(n)$

So easy solution:

2 pointers : front rear } instead of head



deletion

insert

make rear $\rightarrow$ next = newnode

rear = newnode

we did not traverse the list.

So $O(1)$

enqueue(5)
enqueue(0)
enqueue(6)
display()
peek()

struct node

{
int data
struct node *next

}

struct node *front = NULL
* rear = NULL } Q is empty

Enqueue(x)

struct node *newnode.

Create
a node

newnode = (struct node*)

Allocate
mem.

malloc (size of (struct
node));

newnode → data = x

→ next = NULL



newnode

if $f == \text{NULL}$

and

$r == \text{NULL}$

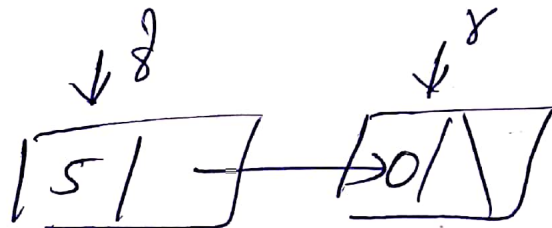
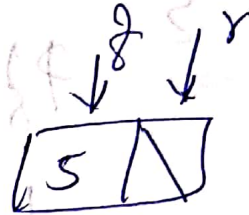
now you have
to insert the
node to Q.

$\Rightarrow$ Q is empty

[$\text{front} = \text{rear} = \text{newnode}$]

else

insert grows rear end.



newnode

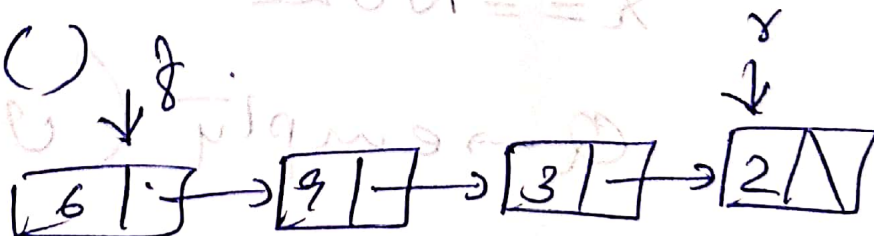
$\rightarrow$ $\text{rear} \rightarrow \text{next} = \text{newnode}$

$\text{rear} = \text{newnode}$

$O(1)$

Egs

display()



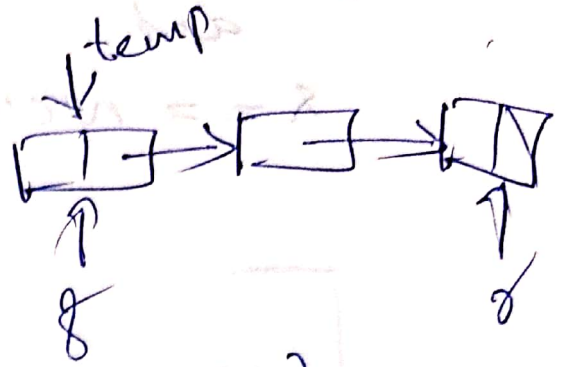
struct node *temp.

if $f == \text{NULL}$ & $r == \text{NULL}$
Q $\rightarrow$ empty

else.

{

temp = front



while (temp != NULL)

{

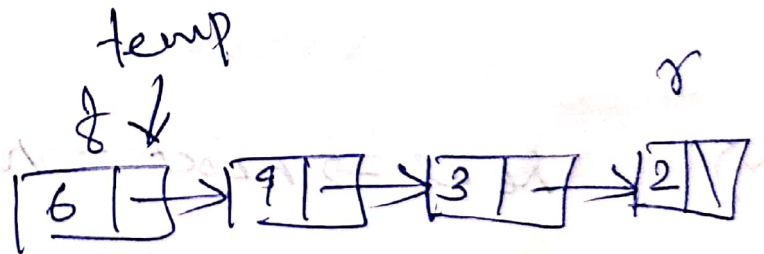
printf("%d", temp->data)

temp = temp->next

}

dequeue()

temp.



if f == NULL & r == NULL

Q → empty (underflow)

else temp = front

front = front->next

free(temp)

peek() front $\rightarrow$ data.

if $f == \text{NULL}$ and
 $r == \text{NULL}$

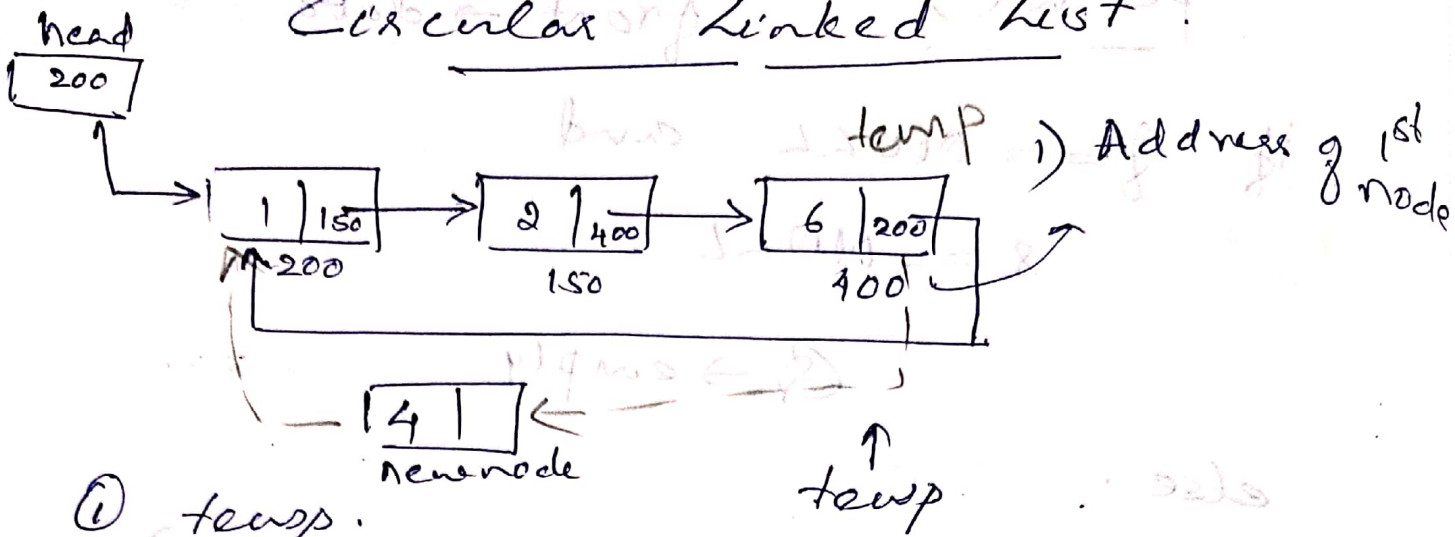
$Q \rightarrow \text{empty}$

else:

printf("%d", front $\rightarrow$ data)



Circular Linked List



① temp.

temp $\rightarrow$ next = newnode.

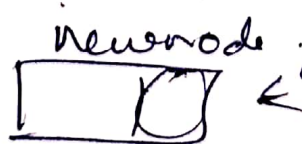
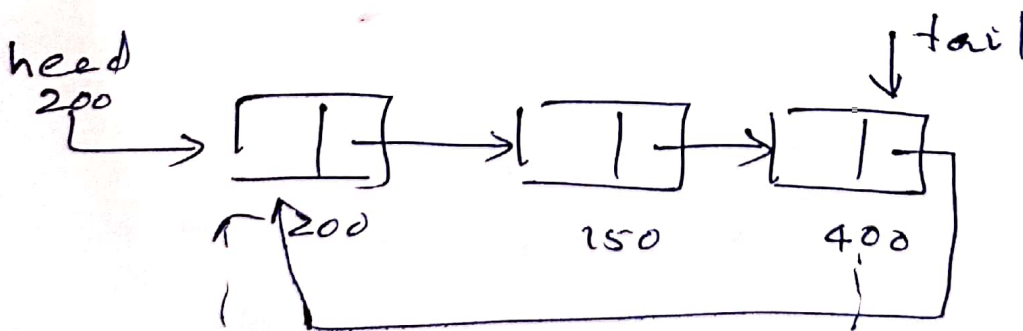
① traverse using temp.

② insert new node.

③ links.

② 2nd method.

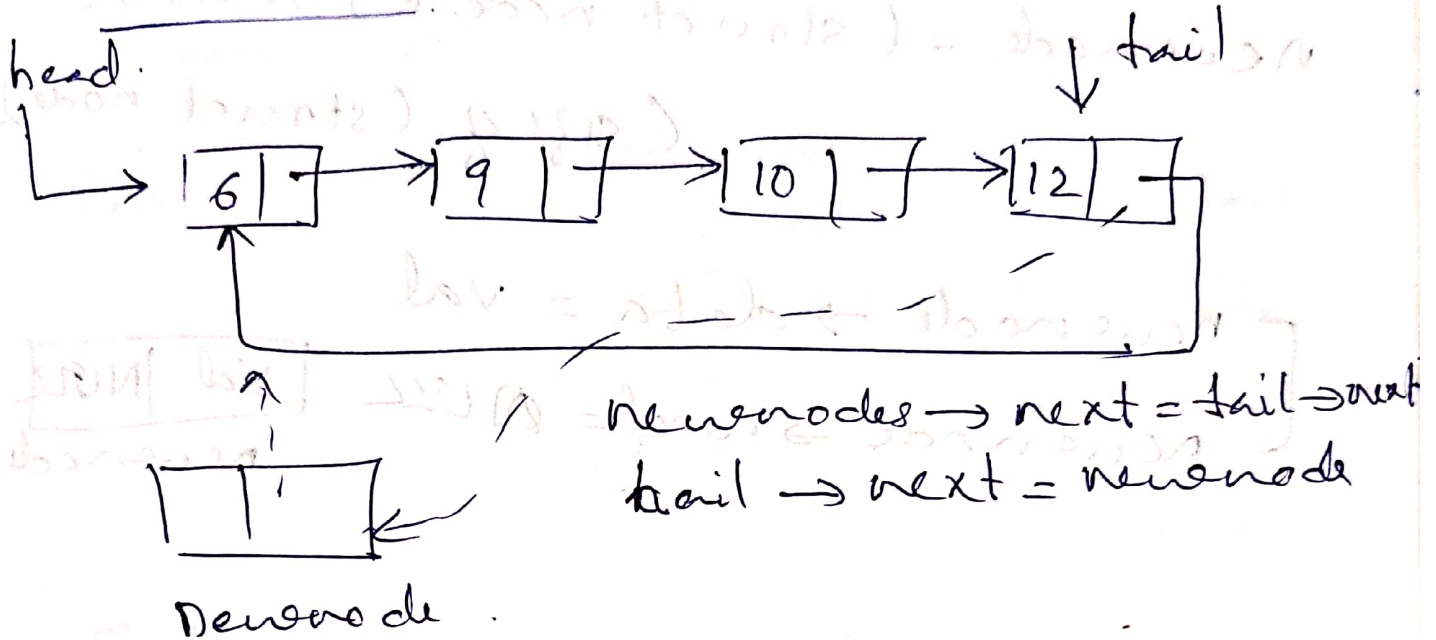
maintain a tail pointer



new node $\rightarrow$ next
= tail $\rightarrow$ next
tail $\rightarrow$ next =
newnode

③ Only tail pointer, ^{need of} no head pointer

Create CLL



* No need of head pointer

struct node

{

int data

struct node * next;

} * head, * tail;

struct node * newnode.

head = NULL.

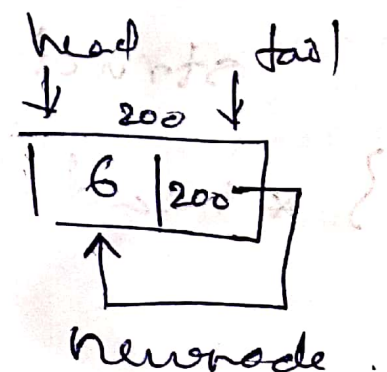
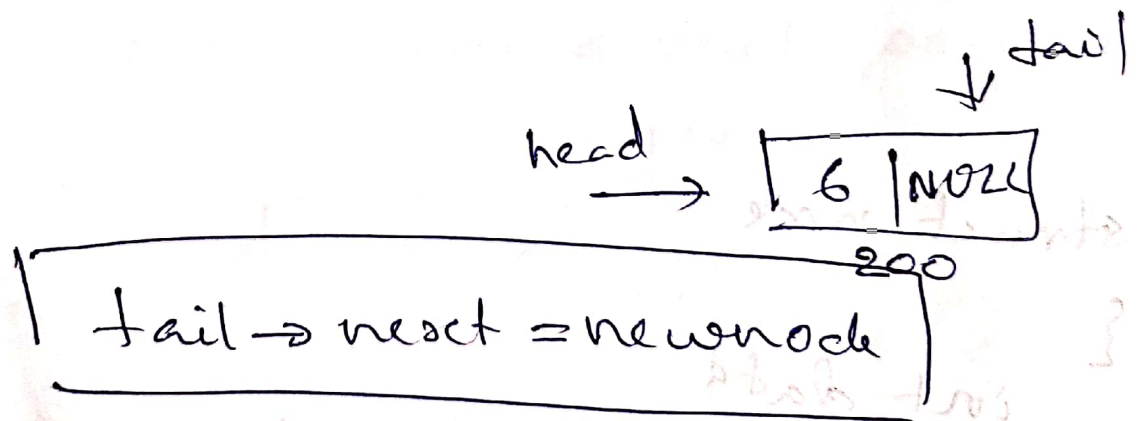
tail = NULL

newnode = (struct node *) malloc
(size of (struct node))

[newnode → data = val
newnode → next = NULL]
Val | NULL
newnode.

→ if head = NULL

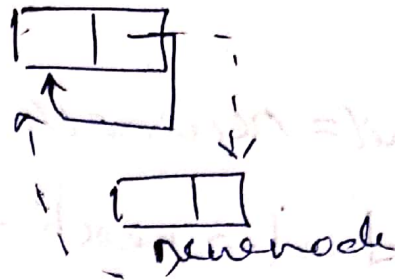
head = tail = newnode



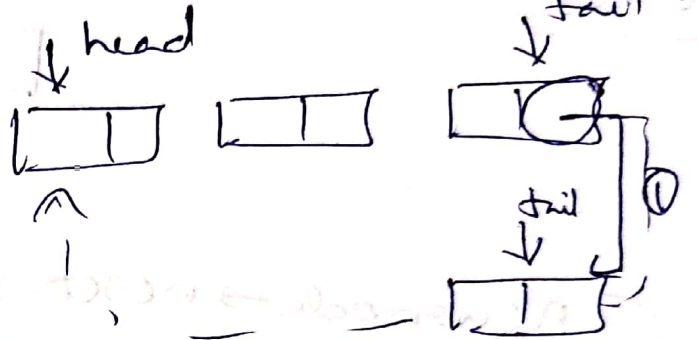
→ else:

~~newnode → next =~~

~~tail → next~~



~~tail → next = newnode~~



tail → next = newnode

tail = newnode

tail → next = head

Only tail pointer

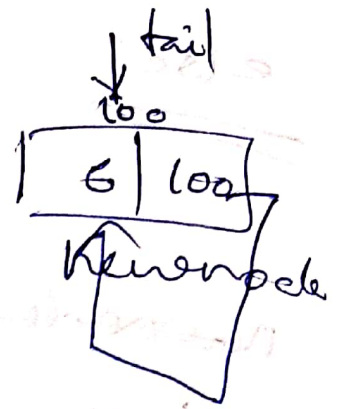
*tail/

tail = NULL

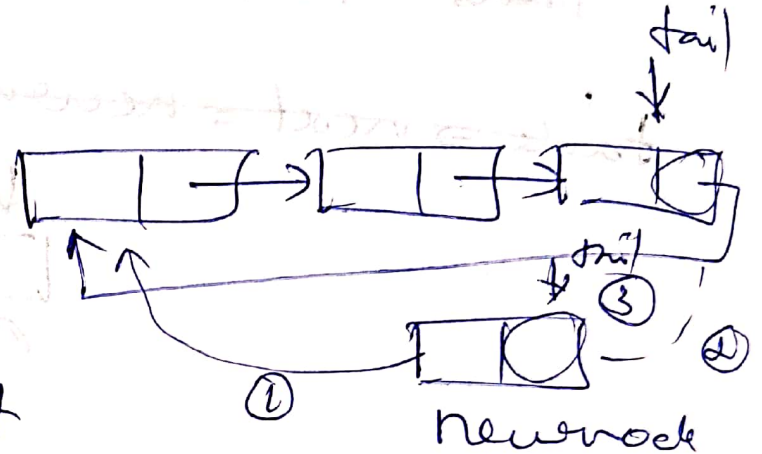
newnode [6 | NULL]

if tail == NULL

{ tail = newnode
tail → next = newnode



else



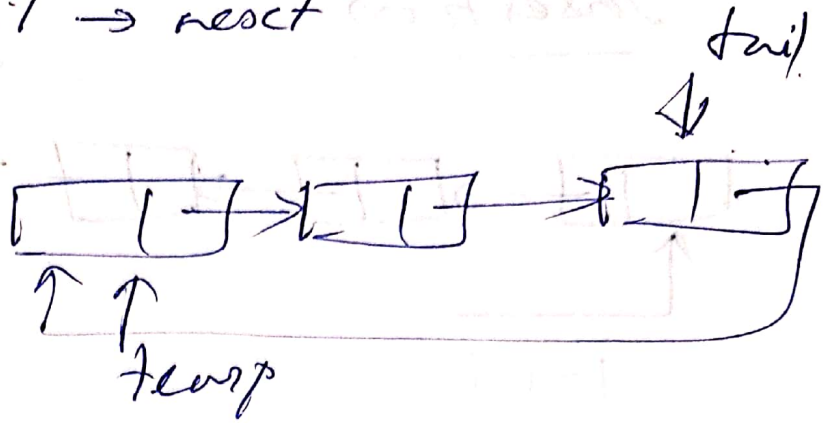
{ newnode → next
= tail → next
tail → next = newnode
tail = newnode

Display

→ we don't know where the list starts.

→ tail → next → starting node address.

temp = tail → next



if tail == NULL

List empty

else

temp = tail → next

temp != tail → next

OR

while (temp → next != tail → next)

[printf("%d", temp → data)
temp = temp → next.

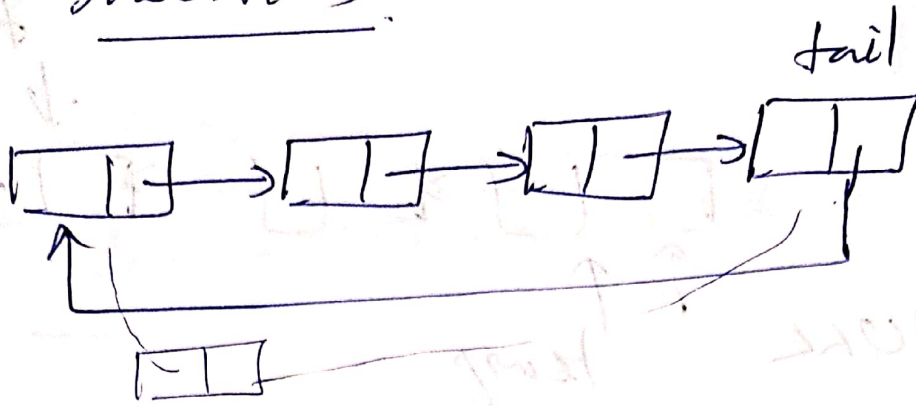
printf("%d", temp → data)

OR do while loop

do
{

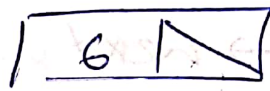
} while (temp != tail → next)

Insertion



at beginning ~~and at end~~

newnode ↓ tail



if $tail == NULL$

$tail = newnode$

~~$newnode \rightarrow next = tail$~~

$tail \rightarrow next = newnode$

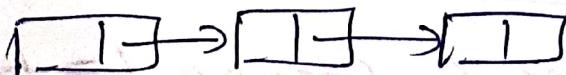
else

$newnode \rightarrow next = tail \rightarrow next$

$tail \rightarrow next = newnode$

~~$tail = newnode$~~

tail need not
be changed
here.



newly inserted one becomes the 1st node now.

at end

if _____

else

newnode $\rightarrow$ next = tail $\rightarrow$ next

tail $\rightarrow$ next = newnode.

tail = newnode

update tail.

at any specific position.

if pos = 1

Insert at beg. of list.

if pos = last

insert at end of list.

Other situations.

$i^0 = 1$
~~*temp~~ $l \rightarrow$ length of list.
pos -

if $pos < 0$ or $pos > l$

invalid positions.

else if $pos == 1$

insert at beg()

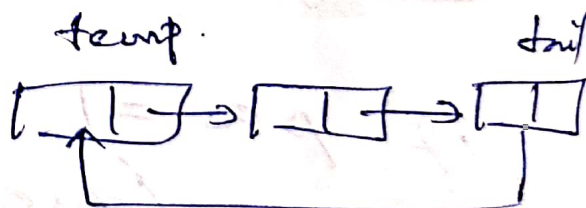
else

traverse till
($pos - 1$)

Create new node



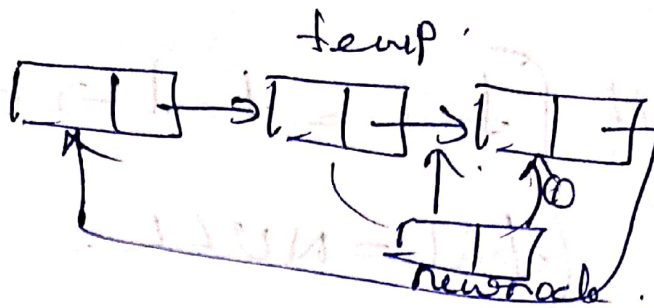
$temp = tail \rightarrow next$



while($i < (pos - 1)$)

$temp = temp \rightarrow next$

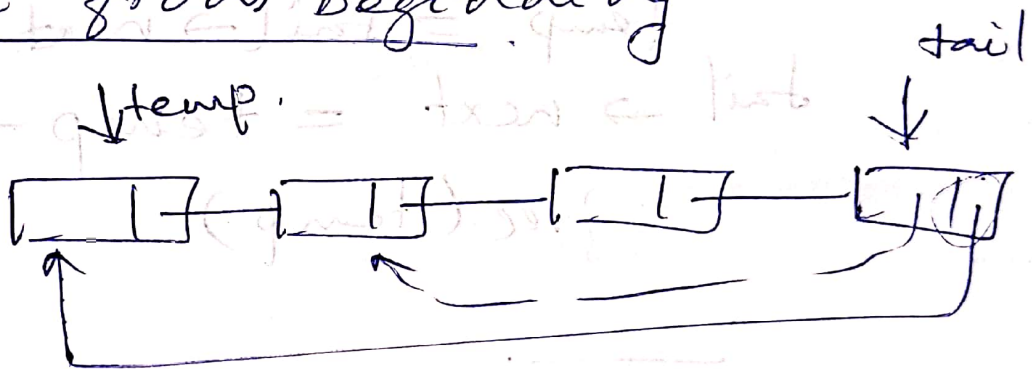
$i++$



$\text{newnode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$
 $\text{temp} \rightarrow \text{next} = \text{newnode}$

Deletion

Delete from beginning



$\text{temp} = \text{tail} \rightarrow \text{next}$

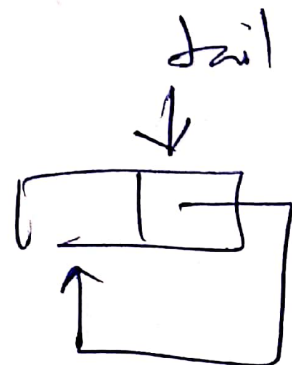
if $\text{tail} == \text{NULL}$
list empty

$\text{tail} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$

$\text{free}(\text{temp})$

1. if $\text{tail} = \text{NULL}$
list empty

2. Only one node in list



else if $\left\{ \begin{array}{l} \text{tail} = \text{tail} \rightarrow \text{next} \\ \text{tail} = \text{NULL}! \end{array} \right.$

$\text{temp} = \text{tail}$

$\text{tail} = \text{NULL}$

~~$\text{tail} = \text{tail} \rightarrow \text{next}$~~ $\text{free}(\text{temp})$

~~Fig~~

else.

$\text{temp} = \text{tail} \rightarrow \text{next}$

$\text{tail} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$

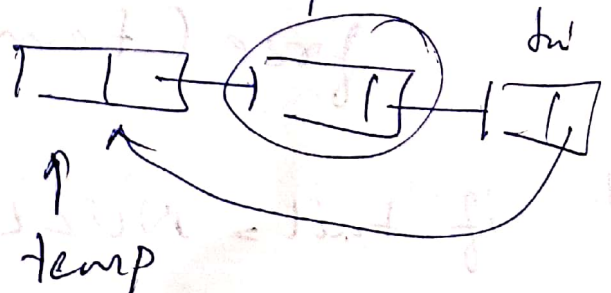
$\text{free}(\text{temp})$

end of list

have to change

tail so.

$\text{temp} = \text{tail} \rightarrow \text{next}$

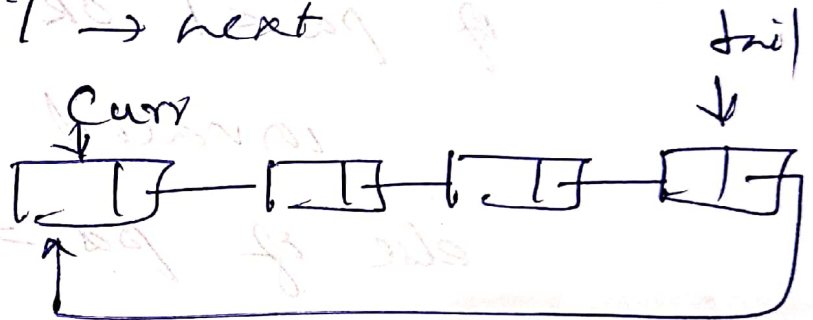


① If list is empty
 $tail == NULL$

② Only one node in list.

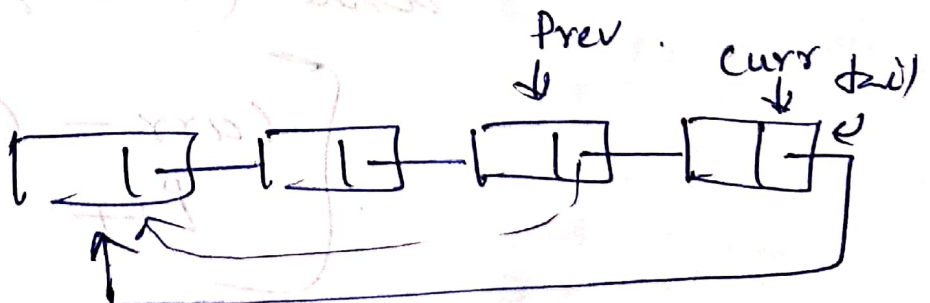
③ * curr
* prev.

$curr = tail \rightarrow next$



while ($curr \rightarrow next \neq tail \rightarrow next$)

prev = curr;
curr → curr → next.



prev → next = tail → next.

tail = prev

free(curr)

Specific position

* curr ~~* prev~~ ~~* temp~~

~~* nextnode~~ temp

pos = 3 i = 1

traverse list till pos - 1

curr = tail → next

~~while (i ≤ pos - 1)~~

{

if pos < 1 OR pos > l.

invalid

else if pos == 1

beg()

else

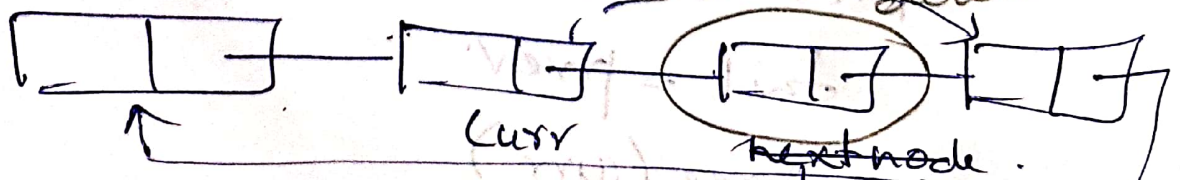
while (i < pos - 1)

curr = curr → next

i++

temp

~~nextnode~~ = curr → next



$\text{curr} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$
 $\text{free}(\text{nextnode})$
 temp

Circular Queue using LL

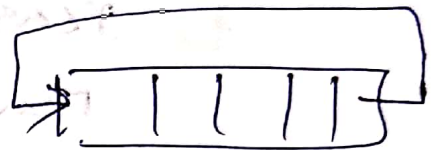
struct node

{ int data;

struct node *next;

} *front = NULL

*rear = NULL



f r.

O(1)

FIFO

Enqueue()

newnode =

newnode $\rightarrow$ data =

$\rightarrow$ next = NULL

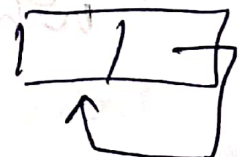
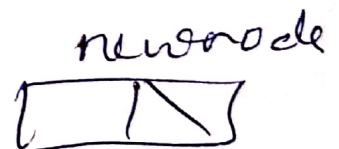
if front == NULL and rear == NULL

Q $\rightarrow$ empty

{ f = r = newnode

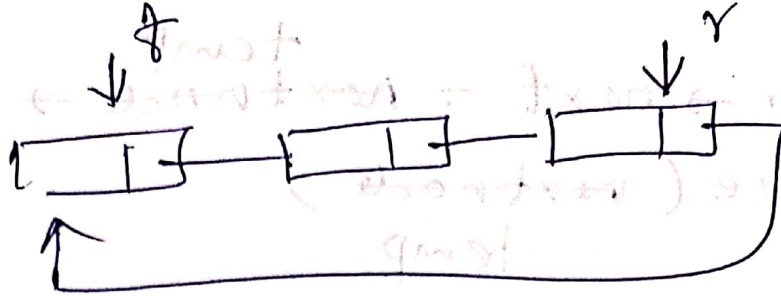
rear $\rightarrow$ next = front.

}



always contains 1st node address

else

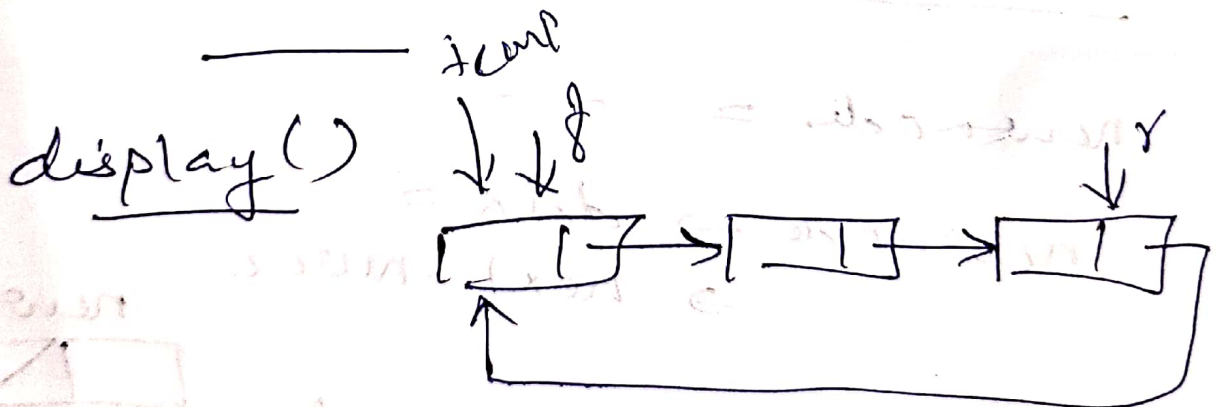
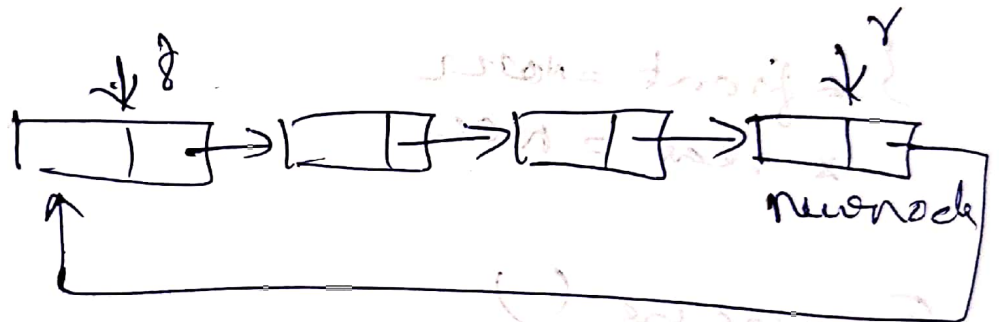


newnode

rear $\rightarrow$ next = newnode

newnode $\rightarrow$ next = front

rear = newnode



~~temp = front~~

~~while (temp $\rightarrow$ next $\neq$ rear $\rightarrow$ next)~~

~~printf("%d", temp $\rightarrow$ data)~~

~~temp = temp $\rightarrow$ next~~

if $f == r == \text{NULL}$

$Q \rightarrow \text{empty}$

else

$\text{temp} = \text{front}$

while ($\text{temp} \rightarrow \text{next} \neq \text{front}$)

$\text{pf}(\text{"\%d", } \&\text{temp} \rightarrow \text{data})$

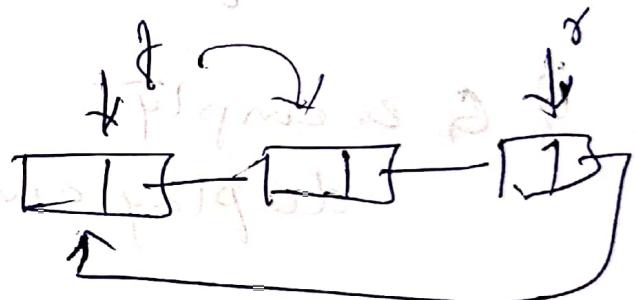
$\text{temp} = \text{temp} \rightarrow \text{next}$

$\text{pf}(\text{"\%d", } \&\text{temp} \rightarrow \text{data})$

Dequeue()

~~temp~~

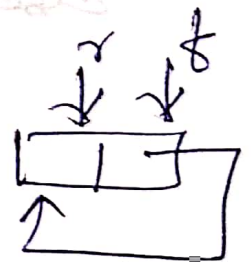
$\text{temp} = \text{front}$



① Q empty - Underflow

if $f == \text{NULL}$

$Q \rightarrow \text{empty}$



②

else if $f == r$

// if one node in Q

$f = r = \text{NULL}$
 $\text{free}(\text{temp})$

③

else

~~$r \rightarrow \text{next}$~~

$f = f \rightarrow \text{next}$
 $r \rightarrow \text{next} = \text{front}$
 $\text{free}(\text{temp})$

peek()

pf (" %d", front $\rightarrow$ data)

if Q is empty
display empty

else

pf (